



Shortest Path in Road Networks Using Dijkstra and Bellman-Ford Algorithms

Final Project Report

CSC 301: Algorithm Analysis and Design

Project Title: *Shortest Path Optimization in Road Networks*

Instructor: *Dr. Rahma Ahmed*

Section Number: *6FA2*

Team Name & Members: *Theta Thinkers*

Name	Student ID
Jana Hassan Algarni	2240004378
Leen Abdulrahman Hadari	2240003905
Asma Abbas Albeshri	2240003453
Lujain Abdullah Alqahtani	2240003619
Alzahraa Hussein Alabbad	2240006089



Table of Contents

1. Introduction.....	3
2. Section I: Theoretical Background	3
2.1 Definition of the Problem	3
2.2 Selected Algorithms.....	4
2.3 Pseudocode of Dijkstra Algorithm.....	4
2.4 Pseudocode of Bellman-Ford Algorithm.....	5
2.5 Literature Review of the Chosen Algorithms	5
2.6 Theoretical Comparison.....	6
3. Section II: Implementation	6
3.1 Dataset Used	6
3.2 How Inputs Were Selected.....	7
3.3 Programming Language and Libraries.....	7
3.4 Implementation Setup	7
3.5 Code for the Chosen Algorithms	8
4. Section III: Analysis and Experimental Results	8
4.1 Computational Complexity by Line Analysis.....	8
4.2 Actual Running Time Results.....	8
4.3 Experimental Analysis.....	9
4.4 Running Time Diagram and Growth Trend.....	10
4.5 Space Complexity Analysis	11
4.6 Comparison with Existing Approaches.....	11
5. Conclusion	12
6. References.....	12
7. Appendices.....	13



1. Introduction

Shortest path optimization is one of the most important problems in algorithm design because it is used in transportation systems, map navigation, routing applications, logistics, and emergency response planning. In a road network, intersections and road endpoints can be represented as vertices, while roads connecting them can be represented as edges. The main goal is to find the minimum-cost route between a source point and a destination point.

This project investigates the shortest path problem using a real road-network dataset. The selected dataset is the California road network, roadNet-CA, from the Stanford Network Analysis Project (SNAP). It represents intersections and endpoints as nodes and roads as undirected edges. The dataset is large enough for performance analysis, with 1,965,206 nodes and 2,766,607 edges according to SNAP.

The project compares two shortest-path algorithms: Dijkstra Algorithm and Bellman-Ford Algorithm. Dijkstra Algorithm follows the greedy design strategy and is efficient for graphs with non-negative edge weights. Bellman-Ford Algorithm is based on repeated relaxation and can handle negative edge weights, although it is usually slower. Since road distances are non-negative, both algorithms can be applied to the same input graph for a fair comparison.

The main objective is to execute both algorithms using the same programming language and dataset, measure their actual execution time, analyze their theoretical time and space complexity, and determine which algorithm is more suitable for solving shortest path problems in road networks. Due to the large size of the full roadNet-CA dataset, a subset of the dataset is used in the implementation experiments while maintaining the same input conditions for both algorithms.

The remaining part of this report is organized as follows. Section I presents the theoretical background, problem definition, pseudocode, literature review, and theoretical comparison. Section II describes the dataset, input selection, programming language, implementation setup, and code execution. Section III presents the experimental analysis, running-time results, order of growth, and diagrams. The report ends with the conclusion, references, and appendices.

2. Section I: Theoretical Background

2.1 Definition of the Problem

Given a graph $G=(V,E)$, where V is the set of vertices and E is the set of edges, the shortest path problem aims to find the path with the minimum total weight between a source vertex s and a destination vertex t . In this project, vertices represent road intersections or endpoints, and edges represent roads connecting them. Since the roadNet-CA dataset is unweighted, each edge is treated as having weight 1, representing one road segment. Therefore, the shortest path means the path with the minimum number of road segments between two selected nodes. Since the dataset is undirected, each road segment can be considered in both directions during implementation. This simplification allows a fair comparison between the two algorithms without relying on real distance values.



2.2 Selected Algorithms

Two algorithms were selected because they solve the same shortest path problem but belong to different algorithmic design families:

Algorithm	Design family	Main idea	Suitable case
Dijkstra	Greedy	Repeatedly chooses the unvisited node with the smallest known distance and relaxes its neighbors.	Graphs with non-negative edge weights.
Bellman-Ford	Relaxation / Dynamic Programming style	Relaxes all edges repeatedly for $ V -1$ iterations to guarantee shortest distances.	Graphs that may contain negative edge weights, as long as there is no negative cycle.

Table 1 Selected Algorithms and Their Design Strategies

Since the dataset does not contain negative weights, Dijkstra is expected to perform better in this project.

2.3 Pseudocode of Dijkstra Algorithm

```

DIJKSTRA(G, source):
  for each vertex v in G:
    distance[v] = infinity
    previous[v] = null
  distance[source] = 0
  create priority queue Q and insert source with priority 0
  while Q is not empty:
    u = vertex in Q with minimum distance
    remove u from Q
    for each neighbor v of u:
      alt = distance[u] + weight(u, v)
      if alt < distance[v]:
        distance[v] = alt
        previous[v] = u
        update priority of v in Q
  return distance, previous

```

Dijkstra Algorithm helps solve the road-network problem by always extending the currently shortest known route. Because road weights are non-negative, the greedy choice is safe and leads to correct shortest paths. In this project, each road segment has weight 1, so Dijkstra Algorithm finds the path with the minimum number of road segments between the selected source and destination nodes.



2.4 Pseudocode of Bellman-Ford Algorithm

```
BELLMAN_FORD(G, source):  
  for each vertex v in G:  
    distance[v] = infinity  
    previous[v] = null  
  distance[source] = 0  
  for i = 1 to |V| - 1:  
    updated = false  
    for each edge (u, v) in E:  
      if distance[u] is not infinity and distance[u] + weight(u, v) < distance[v]:  
        distance[v] = distance[u] + weight(u, v)  
        previous[v] = u  
        updated = true  
    if updated == false:  
      break  
  for each edge (u, v) in E:  
    if distance[u] is not infinity and distance[u] + weight(u, v) < distance[v]:  
      report negative cycle  
  return distance, previous
```

Bellman-Ford Algorithm helps solve the same road-network shortest path problem by repeatedly improving distance estimates through edge relaxation. It is more general than Dijkstra Algorithm because it can handle negative edge weights and detect negative cycles. However, in this project, the roadNet-CA dataset has no negative weights because each road segment is treated as weight 1. Therefore, Bellman-Ford is expected to be less efficient than Dijkstra for this road-network dataset.

2.5 Literature Review of the Chosen Algorithms

Dijkstra algorithm is a classical shortest path algorithm designed for graphs with non-negative edge weights (Dijkstra, 1959). It is widely used in routing and pathfinding because it efficiently finds shortest paths by selecting the currently closest vertex. Bellman-Ford is another well-known single-source shortest path algorithm (Bellman, 1958; Ford, 1956). Unlike Dijkstra, it can process graphs with negative edge weights and can detect negative-weight cycles. However, this additional flexibility comes with higher time complexity.

For road networks, Dijkstra is commonly more practical because road lengths, travel distances, or road segment counts are non-negative. Bellman-Ford is still useful as a comparison algorithm because it demonstrates a different design strategy and highlights the trade-off between generality and efficiency. These differences show how algorithm design affects performance and applicability in real-world scenarios.



2.6 Theoretical Comparison

Algorithm	Best case	Average case	Worst case	Auxiliary Space Complexity	Notes
Dijkstra with priority queue	$O((V + E) \log V)$	$O((V + E) \log V)$	$O((V + E) \log V)$	$O(V)$	Efficient for non-negative weights.
Bellman-Ford with early stopping	$O(E)$	$O(VE)$	$O(VE)$	$O(V)$	More general; supports negative weights and negative-cycle detection.

Table 2 Theoretical Complexity Comparison of Dijkstra and Bellman-Ford Algorithms

The comparison shows that Dijkstra is theoretically more efficient for road networks with non-negative weights, while Bellman-Ford is more general but slower because it may relax all edges repeatedly. If graph storage is included, both algorithms require $O(V + E)$ total space to store the road-network graph.

3. Section II: Implementation

3.1 Dataset Used

The dataset used in this project is roadNet-CA from the Stanford Network Analysis Project (SNAP). It is a California road network where nodes represent intersections or road endpoints, and edges represent roads. The dataset is undirected and contains 1,965,206 nodes and 2,766,607 edges. Since the original dataset is unweighted, each road segment was assigned a weight of 1 in this project.

Due to the very large size of the full roadNet-CA graph, the implementation experiments used a large subset of the dataset. The subset was extracted from the same original roadNet-CA file and was used consistently for both Dijkstra Algorithm and Bellman-Ford Algorithm to ensure a fair comparison.

Dataset item	Description
Dataset name	roadNet-CA
Source	Stanford Network Analysis Project (SNAP)
Graph type	Undirected road network
Nodes	1,965,206
Edges	2,766,607
Meaning of nodes	Road intersections and endpoints
Meaning of edges	Roads connecting intersections or endpoints
Weight assumption	Each edge weight = 1
Implementation subset	A subset of the original roadNet-CA dataset was used for experimental testing due to the large size of the full graph.

Table 3 Summary of the roadNet-CA Dataset

3.2 How Inputs Were Selected

The same input pairs were used for both Dijkstra and Bellman-Ford to ensure a fair comparison. Input pairs consist of a source node and a target node from the road network. Three categories of input sizes/cases were selected:

Case	Input selection method	Purpose
Best case	Source and target are very close or directly connected.	Shows performance when the shortest path is very short.
Average case	Source and target are randomly selected from the same connected component.	Represents typical behavior on normal random inputs.
Worst case	Source and target are far apart in the graph or require many relaxations/search steps.	Shows performance on difficult inputs.

Table 4 Input Selection Cases

For each value of n , a subgraph of size n was extracted from the roadNet-CA subset. The tested input sizes were $n = 100, 500, 1000, 2000$, and 3000 nodes. For each input size, the same source-target pair and the same subgraph were used for both Dijkstra Algorithm and Bellman-Ford Algorithm. The source and target nodes were selected from a connected subgraph to ensure that a valid path exists between them.

3.3 Programming Language and Libraries

Python was used as the programming language for the implementation. The NetworkX library was used to build the graph and execute the shortest-path algorithms, while `time.perf_counter()` was used to measure execution time. The matplotlib library was used to draw the running-time comparison diagram.

Both algorithms were tested using the same programming language, dataset subset, input sizes, and source-target pairs. This ensures that the comparison focuses on the difference between Dijkstra Algorithm and Bellman-Ford Algorithm rather than differences in tools or input data..

3.4 Implementation Setup

Question	Answer to write in the report
What kind of machine did you use?	Windows 11, Intel Core i5, 8 GB RAM.
What timing mechanism?	Python <code>time.perf_counter()</code> was used because it provides high-resolution timing suitable for measuring execution time.
How many times did you repeat each experiment?	Each experiment was repeated 5 times for each input size.
What times are reported?	The average running time in milliseconds was reported for each algorithm, input size, and case type.
Did you use the same inputs for the two algorithms?	Yes. The same source-target pairs and the same subgraphs were used for both algorithms to make the comparison fair.
Screenshots required	Screenshots of running both algorithms on different input sizes, including best, average, and worst cases, will be provided in Appendix B.

Table 5 Implementation Setup Summary



3.5 Code for the Chosen Algorithms

The experimental implementation was written in Python using the NetworkX library. Both Dijkstra Algorithm and Bellman-Ford Algorithm were executed using the same roadNet-CA subset, the same subgraphs, and the same source-target input pairs to ensure a fair comparison.

The code loads the roadNet-CA dataset file, skips comment lines, and extracts the first 50,000 edges to create a practical subset for testing. Since the original dataset is unweighted, each edge is assigned a weight of 1. After building the graph, the largest connected component is selected to ensure that the tested nodes belong to a connected part of the road network.

For each input size, a connected subgraph is extracted from the selected component. Then, source and target nodes are selected according to the case type: best case, average case, or worst case. The same selected source-target pair is used to run both algorithms for that case. The execution time is measured using Python `time.perf_counter()`, and each experiment is repeated five times. The average running time is reported in milliseconds.

The full code is included in Appendix A

4. Section III: Analysis and Experimental Results

4.1 Computational Complexity by Line Analysis

Algorithm	Main operations	$T(n) / T(V,E)$	Order of growth
Dijkstra	Initialize distances, repeatedly extract minimum node from priority queue, relax adjacent edges.	$T(V,E) = O(V \log V + E \log V)$	$O((V + E) \log V)$
Bellman-Ford	Initialize distances, repeat edge relaxation for $V - 1$ iterations.	$T(V,E) = O(VE)$	$O(VE)$

Table 6 Computational Complexity by Line Analysis

The computational complexity of each algorithm is derived by analyzing the main operations in the pseudocode. Dijkstra Algorithm uses a priority queue to repeatedly extract the node with the minimum distance and relax its adjacent edges. This results in a time complexity of $O((V+E)\log V)$. Bellman-Ford Algorithm repeatedly relaxes all edges for up to $V-1$ iterations, resulting in a worst-case time complexity of $O(VE)$. Therefore, Dijkstra Algorithm is theoretically more efficient for road networks with non-negative edge weights.

4.2 Actual Running Time Results

The actual running time was measured using Python `time.perf_counter()`. Each experiment was repeated five times, and the average running time was reported in milliseconds. Both Dijkstra Algorithm and Bellman-Ford Algorithm were tested using the same input sizes, subgraphs, and source-target pairs to ensure a fair comparison. The values in Table 7 represent the average running time for the tested input cases at each input size. The detailed running time results for best, average, and worst cases are provided in Appendix B.



Input size n	Dijkstra avg time (ms)	Bellman-Ford avg time (ms)	Faster algorithm
100	0.1618	0.5348	Dijkstra
500	0.7367	2.4724	Dijkstra
1000	1.6066	4.5053	Dijkstra
2000	2.3973	9.8279	Dijkstra
3000	6.8549	15.7993	Dijkstra

Table 7 Average Running Time Results by Input Size

The results show that Dijkstra Algorithm was faster than Bellman-Ford Algorithm for all tested input sizes. This supports theoretical analysis because Dijkstra Algorithm is more efficient for graphs with non-negative edge weights, while Bellman-Ford Algorithm performs repeated edge relaxation. As the input size increased, the average running time of both algorithms generally increased, but Dijkstra Algorithm consistently achieved lower running time on the selected road-network subset.

4.3 Experimental Analysis

The performance of Dijkstra Algorithm and Bellman-Ford Algorithm was evaluated using different input sizes: $n = 100, 500, 1000, 2000$, and 3000 . The experiments were conducted on connected subgraphs extracted from the roadNet-CA subset. For each input size, the same subgraph and the same source-target pairs were used for both algorithms to ensure a fair comparison.

As shown in Table 7, Dijkstra Algorithm achieved lower average running time than Bellman-Ford Algorithm for all tested input sizes. For small input sizes such as $n = 100$, the difference between the two algorithms is relatively small, where Dijkstra recorded 0.1618 ms and Bellman-Ford recorded 0.5348 ms. However, the difference becomes more noticeable as the input size increases.

Performance Growth

As the input size increases, the running time of both algorithms generally increases. However, Bellman-Ford Algorithm grows more slowly in efficiency because it repeatedly relaxes edges, while Dijkstra Algorithm uses a priority queue to focus on the closest reachable nodes first. For example, at $n = 3000$, Dijkstra Algorithm recorded an average running time of 6.8549 ms, while Bellman-Ford Algorithm recorded 15.7993 ms. This difference is consistent with the theoretical time complexity. Dijkstra Algorithm has a time complexity of $O((V + E) \log V)$, while Bellman-Ford Algorithm has a higher time complexity of $O(VE)$. Therefore, Bellman-Ford requires more repeated operations as the graph size increases.

Graph Interpretation

The running-time diagram shows that Dijkstra Algorithm consistently performs better than Bellman-Ford Algorithm on the selected road-network subset. The graph also shows that the gap between the two algorithms becomes clearer for larger input sizes. This supports the theoretical analysis because the roadNet-CA dataset uses non-negative edge weights, which makes Dijkstra Algorithm more suitable for this problem. Overall, the experimental results demonstrate that Dijkstra Algorithm is more efficient and more practical for shortest path problems in road networks. Bellman-Ford Algorithm remains useful for more general graph problems that may contain negative edge weights, but this advantage is not needed in the selected road-network dataset.

4.4 Running Time Diagram and Growth Trend

Figure 1 shows the experimental running time growth of Dijkstra Algorithm and Bellman-Ford Algorithm as the input size increases. The diagram is based on the measured average running times reported in Table 7. It shows that both algorithms increase in running time as the input size increases. However, Bellman-Ford Algorithm shows a higher growth trend than Dijkstra Algorithm, especially for larger input sizes.

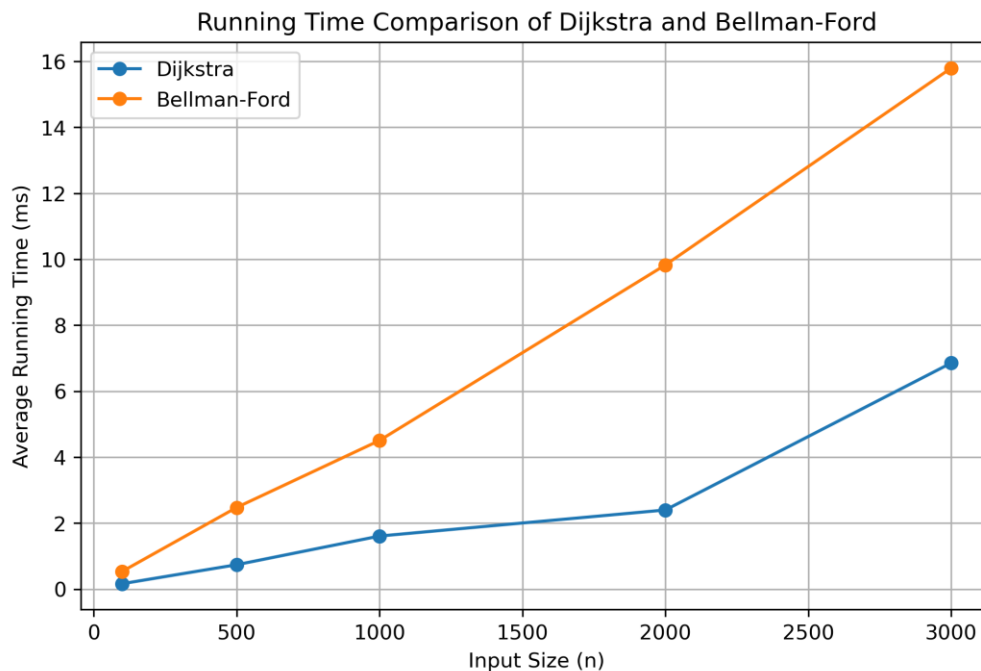


Figure 1 Running Time Comparison of Dijkstra and Bellman-Ford Algorithms

The experimental trend supports the theoretical order of growth. Dijkstra Algorithm has a theoretical time complexity of $O((V+E)\log V)$, while Bellman-Ford Algorithm has a higher time complexity of $O(VE)$. Therefore, the diagram confirms that Dijkstra Algorithm is more efficient for the selected road-network dataset.

Algorithm	Theoretical Order of Growth	Experimental Trend
Dijkstra Algorithm	$O((V + E)\log V)$	Lower growth in running time
Bellman-Ford Algorithm	$O(VE)$	Higher growth in running time

Table 8 Theoretical Order of Growth and Experimental Trend

4.5 Space Complexity Analysis

The space complexity analysis focuses on the memory required to store the graph and the additional data structures used by each algorithm. Since the road network is represented as a graph, both algorithms require memory to store vertices and edges.

Algorithm	Main Memory Usage	Space Complexity
Dijkstra Algorithm	Graph adjacency list, distance dictionary, visited set, and priority queue.	$O(V + E)$
Bellman-Ford Algorithm	Graph structure, distance dictionary, and edge list.	$O(V + E)$ if storing the graph and edge list; $O(V)$ extra memory after graph storage.

Table 9 Space Complexity Analysis

Both algorithms require $O(V+E)$ total space when graph storage is included. However, Dijkstra Algorithm uses an additional priority queue, while Bellman-Ford Algorithm depends mainly on the distance structure and edge list. Since the roadNet-CA dataset is large, graph storage is the main memory requirement for both algorithms.

4.6 Comparison with Existing Approaches

Compared with a general all-pairs shortest path approach such as Floyd-Warshall, both selected algorithms are more suitable for this project because the task is a single-source or source-to-target shortest path problem on a large sparse road network. Floyd-Warshall has $O(V^3)$ time complexity, which is impractical for a graph with millions of nodes.

Dijkstra Algorithm is the most efficient selected algorithm for the roadNet-CA dataset because all road weights are non-negative. Bellman-Ford Algorithm is still correct and more general because it can handle negative edge weights, but it is slower for this type of road-network dataset.

Approach	Time Complexity	Suitable for roadNet-CA?	Reason
Dijkstra Algorithm	$O((V + E)\log V)$	Yes	Efficient on sparse graphs with non-negative weights.
Bellman-Ford Algorithm	$O(VE)$	Possible but slower	Works correctly but has high running time.
Floyd-Warshall Algorithm	$O(V^3)$	No	Designed for all-pairs shortest paths and too expensive for large road networks.

Table 10 Comparison with an Existing Shortest Path Approach

5. Conclusion

This project compared Dijkstra Algorithm and Bellman-Ford Algorithm for solving the shortest path problem on a real road-network dataset. Both algorithms produced correct shortest-path results when applied to the same roadNet-CA subset, subgraphs, and source-target input pairs.

The experimental results show that Dijkstra Algorithm consistently achieved lower running time than Bellman-Ford Algorithm for all tested input sizes. For smaller input sizes, the difference between the two algorithms was relatively limited. However, as the graph size increased, Bellman-Ford Algorithm showed higher running time, while Dijkstra Algorithm remained more efficient.

This behavior matches the theoretical analysis. Dijkstra Algorithm has a time complexity of $O((V + E)\log V)$ when implemented with a priority queue, whereas Bellman-Ford Algorithm has a time complexity of $O(VE)$. Therefore, Bellman-Ford requires more repeated edge relaxation, which makes it slower on larger graphs.

Therefore, Dijkstra Algorithm is more suitable for solving shortest path problems in road networks with non-negative edge weights. Bellman-Ford Algorithm remains useful for graphs with negative weights and for detecting negative cycles, but these advantages are not needed in the selected road-network dataset. As a result, Bellman-Ford is less practical for large-scale road networks compared with Dijkstra Algorithm.

6. References

1. Stanford Network Analysis Project (SNAP). (n.d.). *California road network*. Retrieved from <https://snap.stanford.edu/data/roadNet-CA.html>
2. Stanford Network Analysis Project (SNAP). (n.d.). *Stanford large network dataset collection*. Retrieved from <https://snap.stanford.edu/data/>
3. NetworkX Developers. (n.d.). *Shortest paths*. NetworkX Documentation. Retrieved from https://networkx.org/documentation/stable/reference/algorithms/shortest_paths.html
4. NetworkX Developers. (n.d.). *dijkstra_path*. NetworkX Documentation. Retrieved from https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.shortest_paths.weighted.dijkstra_path.html
5. NetworkX Developers. (n.d.). *bellman_ford_path*. NetworkX Documentation. Retrieved from https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.shortest_paths.weighted.bellman_ford_path.html
6. Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 269–271.
7. Bellman, R. (1958). On a routing problem. *Quarterly of Applied Mathematics*, 16(1), 87–90.
8. Ford, L. R. (1956). *Network flow theory*. RAND Corporation.
9. Floyd, R. W. (1962). Algorithm 97: Shortest path. *Communications of the ACM*, 5(6), 345.
10. Warshall, S. (1962). A theorem on Boolean matrices. *Journal of the ACM*, 9(1), 11–12.



7. Appendices

Appendix A: Full Python Code

The Python implementation file and the dataset file are attached in this appendix. The notebook file Algorithm.ipynb contains the full implementation used to run Dijkstra Algorithm and Bellman-Ford Algorithm. The dataset file roadNet-CA.txt is required to execute the notebook successfully.



Algorithm.ipynb



roadNet-CA.txt

Appendix B: Screenshot of Code Execution

This screenshot shows the detailed running time results for best, average, and worst cases, as well as the summary average running time for each input size. The results confirm that Dijkstra Algorithm achieved lower running time than Bellman-Ford Algorithm for all tested cases.

Detailed Results Table:

Input Size	Case Type	Path Length	Dijkstra Avg Time (ms)	Bellman-Ford Avg Time (ms)	Faster Algorithm
100	Best case	1	0.0276	0.5102	Dijkstra
100	Average case	7	0.1312	0.6224	Dijkstra
100	Worst case	16	0.3265	0.4718	Dijkstra
500	Best case	1	0.0151	2.3593	Dijkstra
500	Average case	8	0.3008	2.4872	Dijkstra
500	Worst case	25	1.8941	2.5705	Dijkstra
1000	Best case	1	0.0273	4.6363	Dijkstra
1000	Average case	20	1.4681	4.297	Dijkstra
1000	Worst case	36	3.3245	4.5826	Dijkstra
2000	Best case	1	0.0258	9.7505	Dijkstra
2000	Average case	15	1.4559	9.4339	Dijkstra
2000	Worst case	37	5.7102	10.2994	Dijkstra
3000	Best case	1	0.0247	16.05	Dijkstra
3000	Average case	32	7.6073	15.1616	Dijkstra
3000	Worst case	72	12.9328	16.1863	Dijkstra

Summary Results Table:

Input Size	Dijkstra Avg Time (ms)	Bellman-Ford Avg Time (ms)	Faster Algorithm
100	0.1618	0.5348	Dijkstra
500	0.7367	2.4724	Dijkstra
1000	1.6066	4.5053	Dijkstra
2000	2.3973	9.8279	Dijkstra
3000	6.8549	15.7993	Dijkstra



Appendix C: Result Tables and Chart Data

This appendix presents the detailed timing results used to calculate the average running time for each input size. The table includes best, average, and worst cases for both algorithms.

Input Size (n)	Case Type	Path Length	Dijkstra Avg Time (ms)	Bellman-Ford Avg Time (ms)	Faster Algorithm
100	Best case	1	0.0276	0.5102	Dijkstra
100	Average case	7	0.1312	0.6224	Dijkstra
100	Worst case	16	0.3265	0.4718	Dijkstra
500	Best case	1	0.0151	2.3593	Dijkstra
500	Average case	8	0.3008	2.4872	Dijkstra
500	Worst case	25	1.8941	2.5705	Dijkstra
1000	Best case	1	0.0273	4.6363	Dijkstra
1000	Average case	20	1.4681	4.2970	Dijkstra
1000	Worst case	36	3.3245	4.5826	Dijkstra
2000	Best case	1	0.0258	9.7505	Dijkstra
2000	Average case	15	1.4559	9.4339	Dijkstra
2000	Worst case	37	5.7102	10.2994	Dijkstra
3000	Best case	1	0.0247	16.0500	Dijkstra
3000	Average case	32	7.6073	15.1616	Dijkstra
3000	Worst case	72	12.9328	16.1863	Dijkstra

Table 11 Detailed Running Time Results by Case Type